

Pushdown Normal Form Bisimulation

A Nominal, Context Free Approach to program equivalence

By Fares Mestre, based on the paper by Vaselios Koutavas, Yuyang Lin, and Nikos Tzevelekos [1]

That's a lot of big words

Pushdown

Normal-Form

Bisimulation

As in “Pushdown Automata”

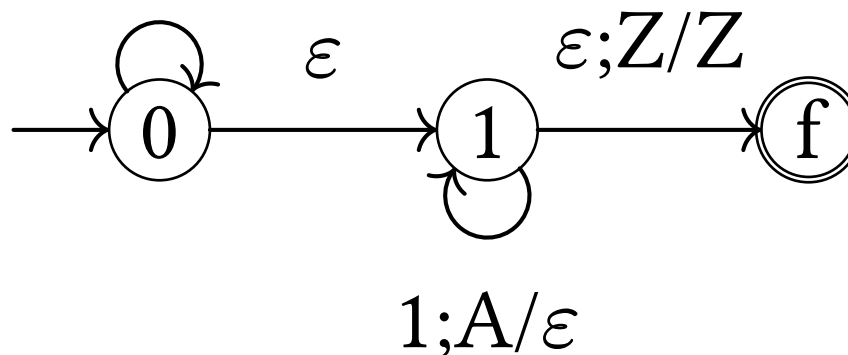
Pushdown

Normal-Form

Bisimulation

Pushdown automaton for language $\{0^n 1^n \mid n \in \mathbb{N}\}$

$0;Z/AZ, 0;A/AA$

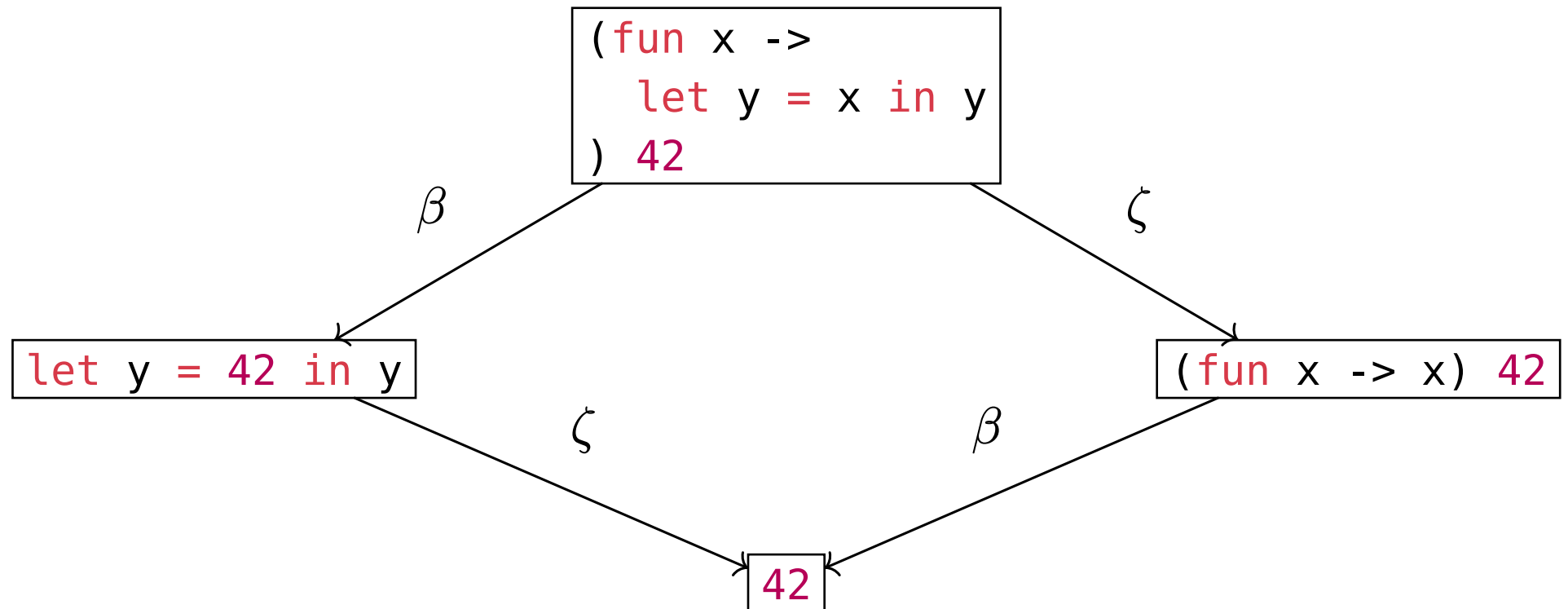


The Normal Form of an expression

Pushdown

Normal-Form

Bisimulation



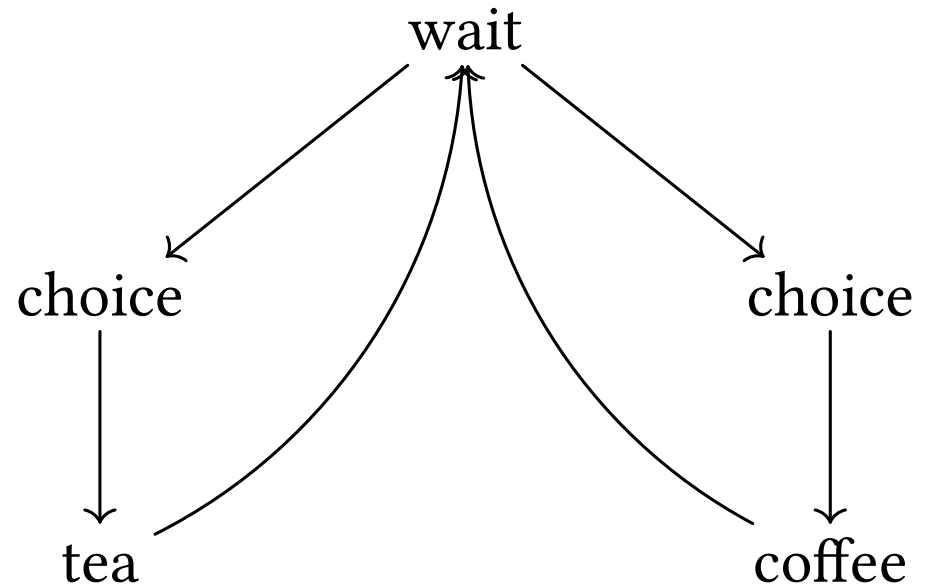
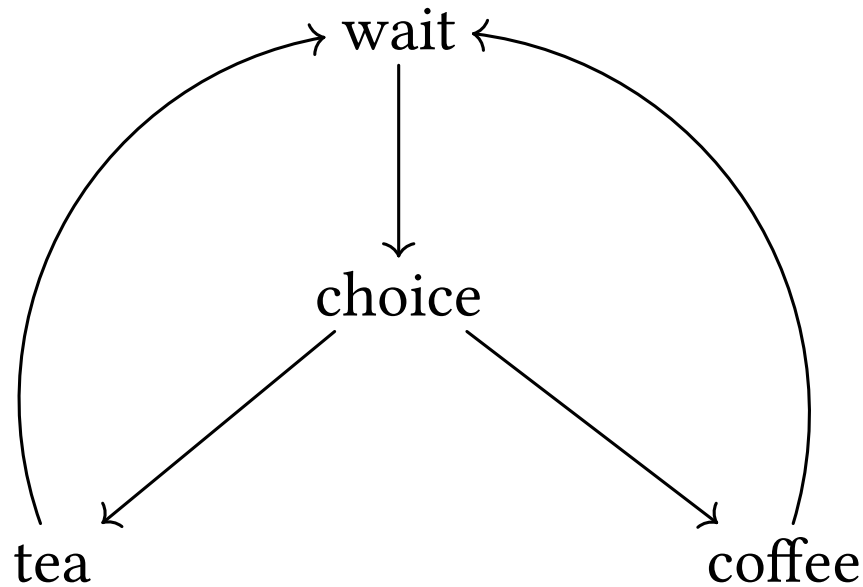
Bisimulation is a form of program equivalence

Pushdown

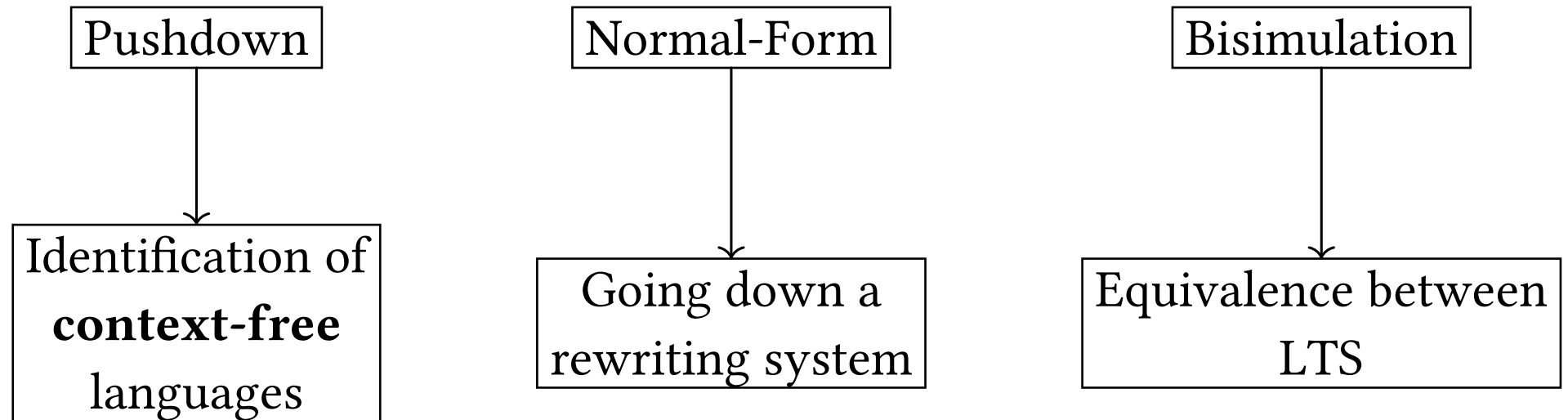
Normal-Form

Bisimulation

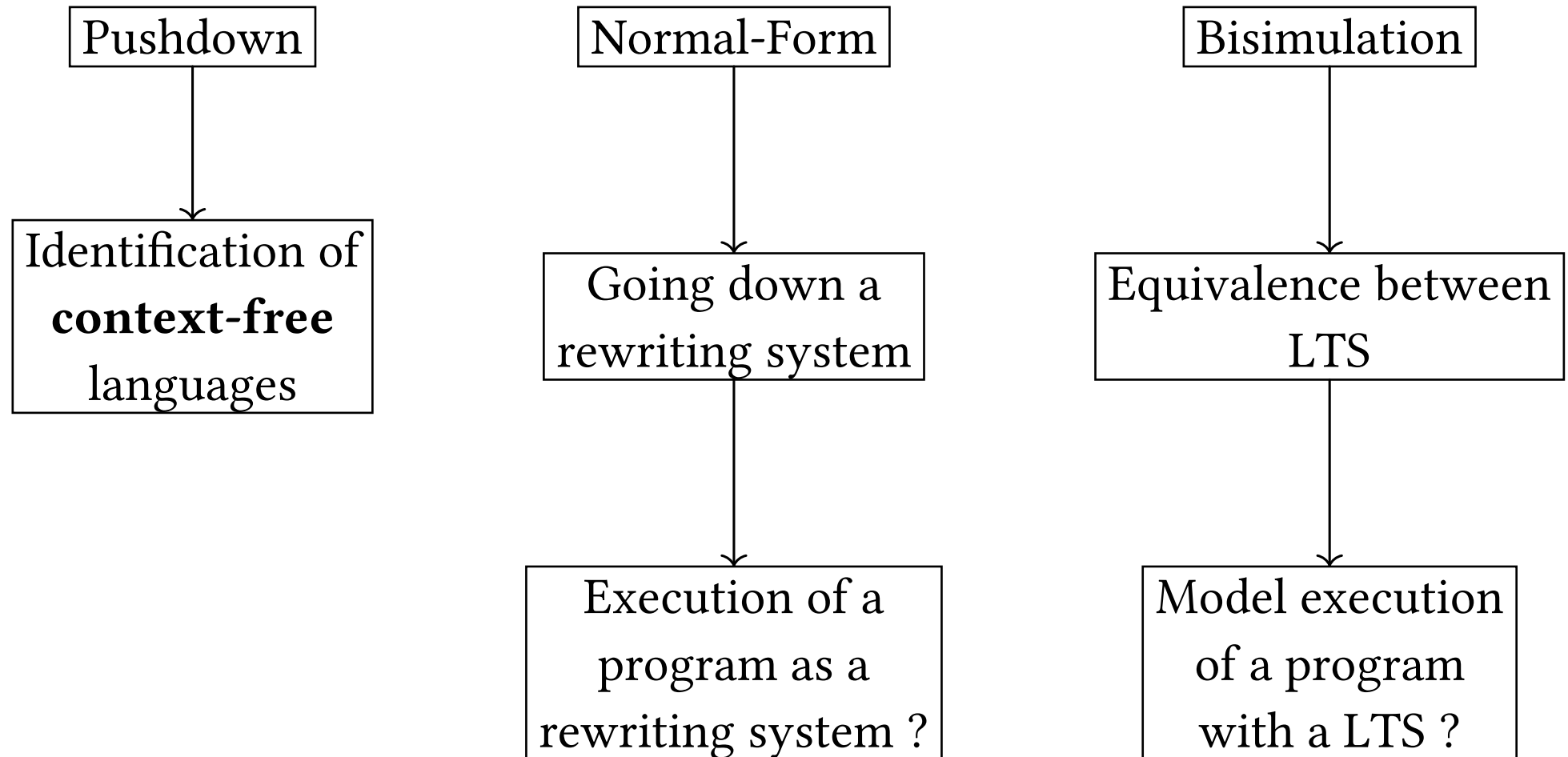
Let's play a game



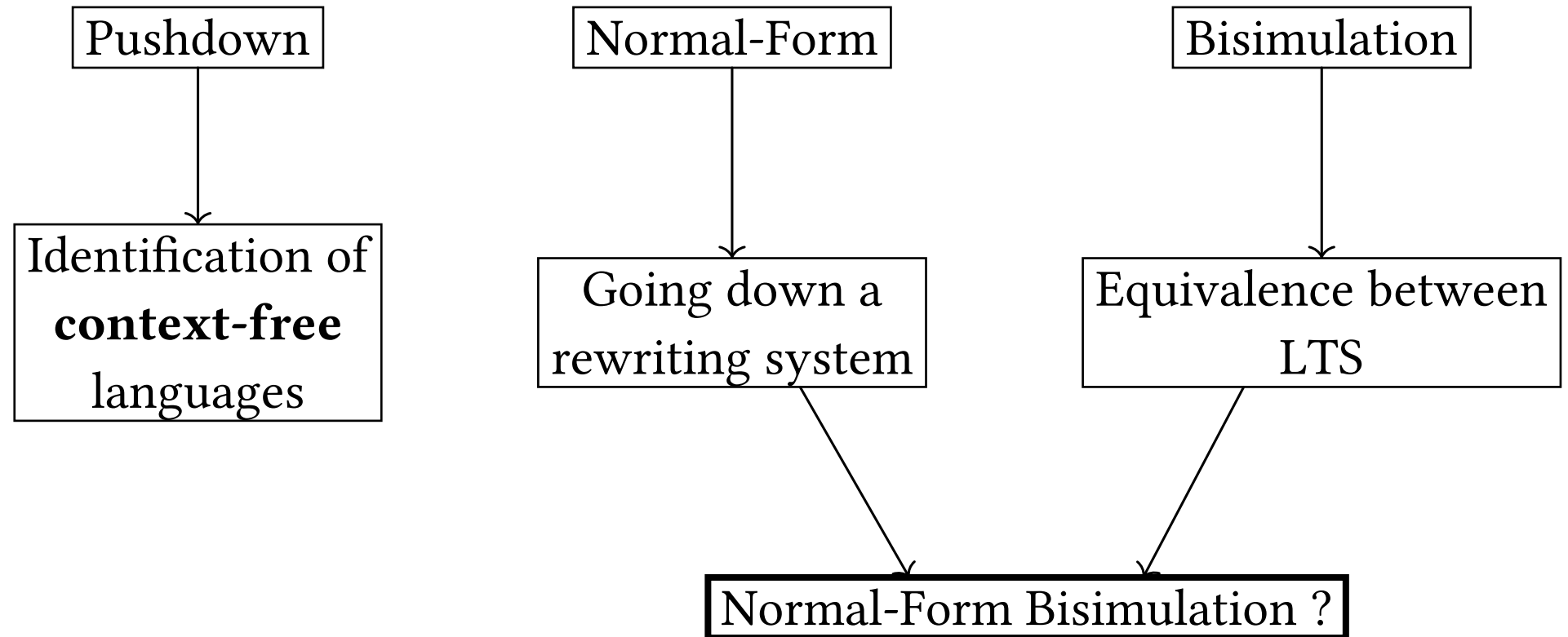
How are these terms related ?



How are these terms related ?



How are these terms related ?



Normal-Form Bisimulation

In a vacuum

- Introduced to prove semantic equivalence on lazy λ -calculus [2]
- It has then been expanded to fit other commonplace programming concepts
 - Polymorphism
 - Recursive types
 - Call-by-name/value
 - etc...

We are trying to extend the concept to arbitrary function calls with side-effects

Normal-Form Bisimulation

A higher-order example

```
let createElement
  (onstart : unit -> unit)
  (onend : unit -> unit) =
  let ref flag = false in
  let event () =
    flag := true ;
    onstart () ;
    flag := false ;
    onend () ;
    !flag
  in event
```

```
let createElement
  (onstart : unit -> unit)
  (onend : unit -> unit) =
  let event () =
    onstart () ;
    onend() ;
    false
  in event
```

We don't know what goes on in the argument functions!

Studied language

λ^{imp} : a simply typed lambda-calculus with local-state

- ML-like syntax
- Functions may be recursive ($\text{fix } f(x).e$) or abstract (α)
 - Abstract functions have a set type $T \rightarrow T'$
 - Their names are in the countably infinite set ANam
- Uses small-step semantics between **store** and expression
 $(\langle s; e \rangle \rightarrow \langle s'; e' \rangle)$
- Has simple types (unit, int, bool, functions & tuples) with the type judgements one would expect
 - Type judgements are denoted with $\Delta; \Lambda \vdash e : T$
 - Δ is a type environment
 - Λ is a **store** typing

Syntax 1 Semantics 1

Studied language

How it handles stateful operations

- **Stores** are a mapping between locations and closed values
 - These values are uniquely typed ; hence a store typing Λ
- Contexts may contain typed, (non-uniquely) indexed holes $[\cdot]_{i,T}$
 - We only really use **canonical** contexts, whose holes are indexed 1...n
 - $D[\vec{e}]$ denotes a **canonical** context with each hole filled by e_i
- Environments Γ map indices to closed values
 - $D[\Gamma]$ denotes a context with each hole filled by $\Gamma(i)$

Syntax 1

Modeling NF-Bisimulation

A model of the bisimulation game

Proponent configuration

$\langle \Gamma ; K ; s ; e \rangle$

Opponent configuration

$\langle \Gamma ; K ; s ; \xi \rangle$

- Γ is an environment indexing proponent functions known to opponent
- ξ is a continuation ; it is either empty ($\diamond$) or the most recent evaluation context
 - Evaluation Contexts are used to define our small-step semantics

Semantics 1

- K is the stack of continuations ξ created by opponent
- s is the store containing proponent locations
- e is the expression being reduced on proponent configurations
- $\langle \perp \rangle$ represents divergent configurations

Modeling NF-Bisimulation

A model of the bisimulation game

Proponent moves

call (α , D)

ret (D)

Opponent moves

call (i, D[$\vec{\alpha}$])

ret (D[$\vec{\alpha}$])

- There are rules to playing these moves
 - We will call them **proponent** and **opponent** properties respectively
 - A move cannot be played if the conditions aren't met
 - However, we still need to introduce new notions to express those rules

Player Properties

Contexts

- We only work on certain contexts $D^\bullet$
 - A constant c
 - A hole $[\cdot]_{i, T \rightarrow T'}$
 - A syntactic sequence $\overrightarrow{D^\bullet}$
- These contexts have an associated signature $\text{sig}(D^\bullet)$
 - $\text{sig}(c) = \varepsilon$
 - $\text{sig}([\cdot]_{i, T \rightarrow T'}) = i$
 - $\text{sig}(\overrightarrow{D^\bullet}) = \overrightarrow{\text{sig}(D^\bullet)}$
- This last definition produces a non-repeating sequence, in which each hole appears exactly once.

Player Properties

Ultimate Patterns

- They are pairs $(D^\bullet, \Gamma)$ associated with a value v
 - Those belong to a set $\text{ulpatt}(v)$

$$(D^\bullet, \Gamma) \in \text{ulpatt}(v) \Leftrightarrow v = D^\bullet[\Gamma] \wedge \text{dom}(\Gamma) = \{i \mid i \in \text{sig}(D^\bullet)\}$$

- This can be extended to types T

$$(D^\bullet, \Gamma) \in \text{ulpatt}(T) \Leftrightarrow \vdash D^\bullet[\Gamma] : T \wedge \Gamma : \{i \mid i \in \text{sig}(D^\bullet) \rightarrow \text{ANam}\}$$

We now have everything we need to express our rules...
except for the moves themselves

Available Moves

Proponent

$$\text{PROPCALL} : \langle \Gamma ; K ; s ; E[\alpha \ v] \rangle \xrightarrow{\text{call } (\alpha, D)} \langle \Gamma, \Gamma' ; K ; s ; E \rangle \quad \text{if } (\mathbf{P})$$

$$\text{PROPRET} : \langle \Gamma ; \xi, K ; s ; v \rangle \xrightarrow{\text{ret } (D)} \langle \Gamma, \Gamma' ; K ; s ; \xi \rangle \quad \text{if } (\mathbf{P})$$

Proponent property :

$$\mathbf{P} : (D, \Gamma') \in \text{ulpatt}(v)$$

- We destructure values v as $D[\Gamma]$
- We abstract away abstract function α calls into indices in environment
- New environment gets added after **proponent** plays

Available Moves

Opponent

$$\text{OPPCALL} : \langle \Gamma ; K ; s ; \xi \rangle \xrightarrow{\text{call } (i, D[\vec{\alpha}])} \langle \Gamma ; \xi, K ; s ; e \rangle \quad \text{if } (\mathbf{O}')$$

$$\text{OPPRET} : \langle \Gamma ; K ; s ; E[\cdot]_T \rangle \xrightarrow{\text{ret } (D[\vec{\alpha}])} \langle \Gamma ; K ; s ; E[D[\vec{\alpha}]] \rangle \quad \text{if } (\mathbf{O})$$

Opponent property :

$$\mathbf{O} : \vec{\alpha} \# \Gamma, K, s, e, \xi \wedge (D, \vec{\alpha}) \in \text{ulpatt}(T)$$

$$\mathbf{O}' : \mathbf{O} \wedge \Lambda_s \vdash \Gamma(i) : T \rightarrow T' \wedge \Gamma(i) D[\vec{\alpha}] \succ e$$

- We pass abstract functions $\vec{\alpha}$ to the **proponent**
- But we still need more notation...

Available Moves

Notation in **opponent** properties

$$\mathbf{O} : \vec{\alpha} \# \Gamma, K, s, e, \xi \wedge (D, \vec{\alpha}) \in \text{ulpatt}(T)$$

$$\mathbf{O}' : \mathbf{O} \wedge \Lambda_s \vdash \Gamma(i) : T \rightarrow T' \wedge \Gamma(i) D[\vec{\alpha}] \succ e$$

- Notation : $\vec{\alpha} \# \vec{X} \Leftrightarrow \forall i, j, \alpha_i \notin \text{an}(X_j)$
“None of the abstract names in $\vec{\alpha}$ are in any component of $\vec{X}$ ”
- Reminders
 - $\Gamma(i)$ returns an abstract function α
 - $D[\vec{\alpha}]$ denotes a canonical context with each hole $[\cdot]_{i,T}$ filled with α_i
- $vu \succ e$: depends on v
 - $(\text{fix } f(x).e)u \succ e[u/x][\text{fix } f(x).e/f]$ β -reduction
 - $\alpha u \succ \alpha u$

Available Moves

Internals

TAU :

$$\langle \Gamma ; K ; s ; e \rangle \xrightarrow{\tau} \langle \Gamma ; K ; s' ; e' \rangle \quad \text{if } \langle s, e \rangle \rightarrow \langle s', e' \rangle$$

RESPONSE :

$$C \longrightarrow \langle \perp \rangle \quad \text{if } C \text{ cannot use any of the other rules}$$

- τ in the context of LTS often denotes an internal system transformation
 - Here, it's the application of the small-step semantics of λ^{imp}
 - We internally rewrite before playing : hence **Normal Form** Bisimulation
- $\langle \perp \rangle$ represents a divergent configuration

A motivating example

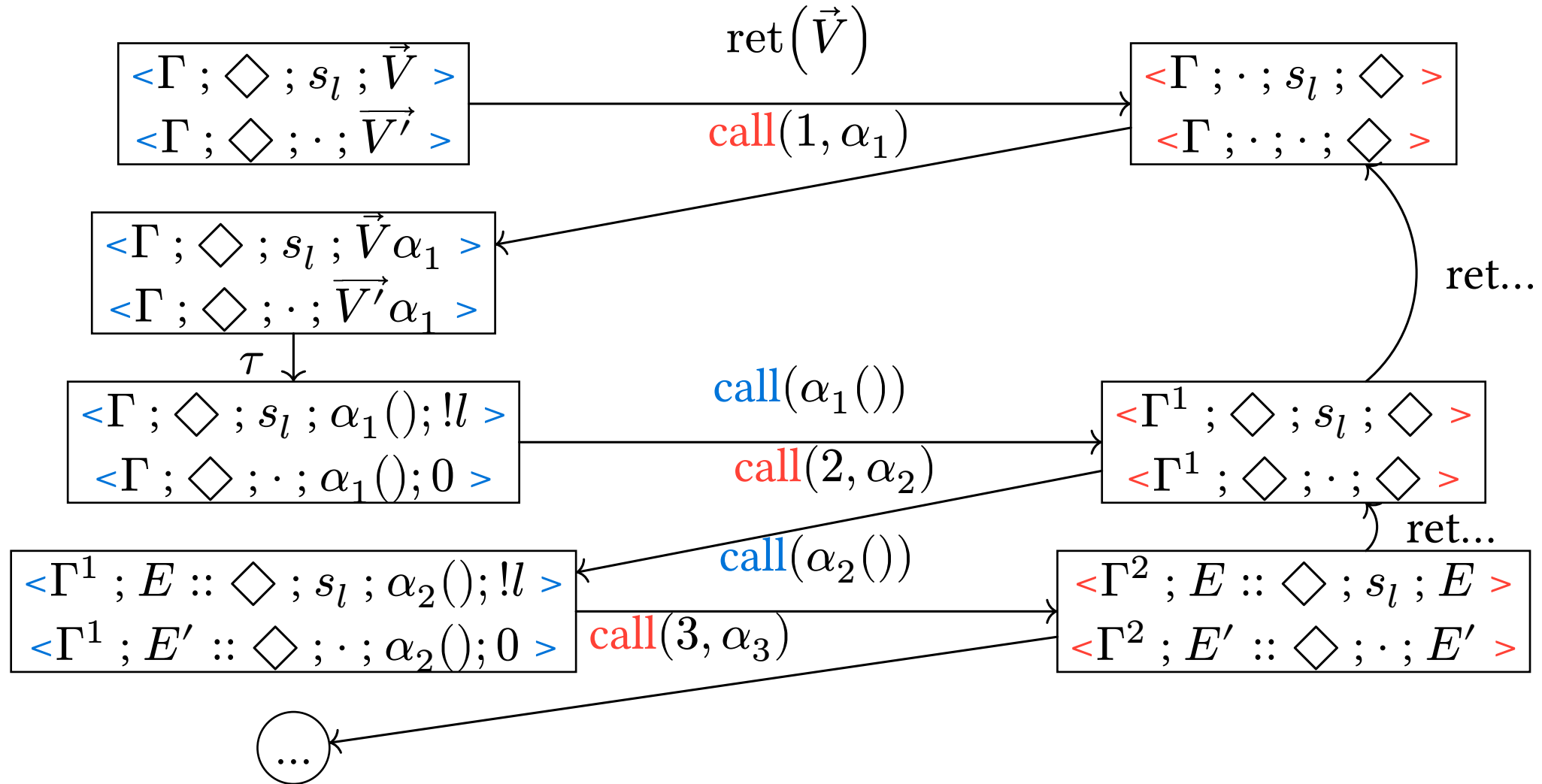
- We want to perform NF-Bisimulation on two programs

$$\vec{V} = \text{fix } f_{\text{unit} \rightarrow \text{unit}}(x).fx;!l$$

$$\overrightarrow{V'} = \text{fix } f_{\text{unit} \rightarrow \text{unit}}(x).fx;0$$

- A bit of notation before showing the NF-Bisimulation LTS :
 - ▶ Γ is our initial environment
 - Γ^n denotes $\Gamma^{n-1}, [n \mapsto \alpha_n]$
 - ▶ s_l is the store containing the value of l
 - ▶ E denotes the continuation of $V : [-];!l$ (resp. E')

A motivating example



A motivating example

- On paper, we're good !
 - So long as we can prove which player wins, we can have a decisive result

BUT!

What if we wanted to implement such a Labeled Transition System ?

Infinite function calls in a limited space → **stack overflow...**

The HOBBIT by J.R.R Tolkien

- Critically acclaimed fantasy novel
- Adapted into a resounding success of a movie trilogy
- I'm going to be-lbo Bagging an F if I keep going off-topic

HOBbit

Higher- Order Bounded Bisimulation Tool

- Verifies program equivalences
- Finds inequivalences
- Combines
 - Bounded Model-Checking
 - Up-To techniques

What's Up-To techniques ? A brief summary :

- Separation : eliminate names in Γ if they manipulate local refs only
- Generalization : generalize store values using a given invariant
- Reentry : prevent reentry of functions if store has not changed between calls

Some more results on NF-Bisimulation

- Compatibility : let C_1 and C_2 two configurations. They are compatible when either
 - At least one is $< \perp >$
 - They have the same polarity **and** their environments $C_1.\Gamma$ and $C_2.\Gamma$ have the same domain
- Let $C_1 RC_2$ a relation between compatible configurations. It is a **weak simulation** if
 - $C_1 = < \Gamma ; \cdot ; s ; \Diamond > \Rightarrow C_2 = < \Gamma ; \cdot ; s ; \Diamond >$
 - $C_1 \xrightarrow{\eta} C_1' \text{ with } \vec{\alpha} \# C_2 \Rightarrow C_2 \xrightarrow{\tau^*; \eta; \tau^*} C_2' \text{ and } C_1' RC_2'$
- If both R and R° are weak simulations, R is a **bisimulation**
- **Similarity** $\left(\begin{smallmatrix} \sqsubseteq \\ \approx \end{smallmatrix} \right)$ is the largest weak simulation, so is **Bisimilarity** ($\approx$)

Some more results on NF-Bisimulation

- Contextual equivalence : let e_1 and e_2 two expressions with no abstract name.

If, for all contexts D such that $\vdash D[e_1] : \text{unit}$ and $\vdash D[e_2] : \text{unit}$,

$$\langle \cdot, D[e_1] \rangle \xrightarrow{*} \langle t, v \rangle \Leftrightarrow \langle \cdot, D[e_2] \rangle \xrightarrow{*} \langle t, v \rangle$$

for some $\langle t, v \rangle$. We denote $e_1 \equiv e_2$

- Bisimilarity between expressions : let $\vdash e_1 : T$ and $\vdash e_2 : T$ with no abstract name, e_1 is **bisimilar** to e_2 ($e_1 \approx e_2$) when

$$\langle \cdot; \cdot; \cdot; e_1 \rangle \approx \langle \cdot; \cdot; \cdot; e_2 \rangle$$

Some more results on NF-Bisimulation

We get a result close to the Hennessy-Milner theorem for CTL-distinguishability on Kripke Systems

Hennessy-Milner Theorem :

$$K_1 \approx K_2 \Leftrightarrow K_1 \equiv_{\text{CTL}} K_2$$

Our theorem : [3]

$$e_1 \approx e_2 \Leftrightarrow e_1 \equiv e_2$$

So we can make use of previous works on contextual equivalence to have bisimilarity!

Conclusion



Fireproof-cats

PDFN-Bisimulation

Bibliography

- [1] V. Koutavas, Y.-Y. Lin, and N. Tzevelekos, “Pushdown Normal-Form Bisimulation: A Nominal Context-Free Approach to Program Equivalence,” in *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*, in LICS '24. Tallinn, Estonia: Association for Computing Machinery, 2024. doi: 10.1145/3661814.3662103.
- [2] D. Sangiorgi, “The Lazy Lambda Calculus in a Concurrency Scenario,” *Information and Computation*, vol. 111, no. 1, pp. 120–153, 1994, doi: <https://doi.org/10.1006/inco.1994.1042>.
- [3] V. Koutavas, Y.-Y. Lin, and N. Tzevelekos, “From Bounded Checking to Verification of Equivalence via Symbolic Up-to Techniques,” in *Tools and Algorithms for the Construction and Analysis of Systems*, D. Fisman and G. Rosu, Eds., Cham: Springer International Publishing, 2022, pp. 178–195.

λ^{imp} syntax

Loc : l, k Var : x, y, z Const : c ANam : α

Type: $T ::= \text{bool} \mid \text{int} \mid \text{unit} \mid T \rightarrow T' \mid T_1 * \dots * T_n$

Exp : $e, M ::= v \mid (\vec{e}) \mid \text{op}(\vec{e}) \mid ee \mid \text{if } e \text{ then } e \text{ else } e \mid \text{ref } l = v \text{ in } e$
 $\mid l := e \mid \text{let}(\vec{x}) = e \text{ in } e$

Val : $u, v ::= c \mid x \mid \alpha_{T \rightarrow T'} \mid \text{fix } f_{T \rightarrow T'}(x).e \mid (\vec{v})$

ECtx : $E ::= [\cdot]_T \mid (\vec{v}, E, \vec{e}) \mid \text{op}(\vec{v}, E, \vec{e}) \mid Ee \mid vE \mid \text{if } E \text{ then } e \text{ else } e$
 $\mid l := E \mid \text{let}(\vec{x}) = E \text{ in } e$

Ctx : $D ::= [\cdot]_{i,T} \mid e \mid (\vec{D}) \mid \vec{D} \mid DD \mid \text{if } D \text{ then } D \text{ else } D$

$\mid \text{ref } l = D \text{ in } D \mid l := D \mid \text{let}(\vec{x}) = D \text{ in } D \mid \text{fix } f_{T \rightarrow T'}(x).D$

Section Section

λ^{imp} reduction semantics

$$\langle s, E[e] \rangle \rightarrow \langle s', E[e'] \rangle$$

$$\text{if } \langle s, e \rangle \hookrightarrow \langle s', e' \rangle$$

$$\langle s, \vec{c} \rangle \hookrightarrow \langle s, w \rangle$$

$$\text{if } \text{op}^{\text{arith}}(\vec{c}) = w$$

$$\langle s, (\text{fix } f(x).e)v \rangle \hookrightarrow$$

$$\langle s, e[v/x][\text{fix } f(x).e/f] \rangle$$

$$\langle s, \text{if } c \text{ then } e_1 \text{ else } e_2 \rangle$$

$$(c, i) \in \{(\text{tt}, 1), (\text{ff}, 2)\}$$

$$\langle s, \text{let } (\vec{x}) = (\vec{v}) \text{ in } e \rangle \hookrightarrow \langle s, e[\vec{v}/\vec{x}] \rangle$$

$$\langle s, \text{ref } l = v \text{ in } e \rangle \hookrightarrow \langle s[l \mapsto v], e \rangle$$

$$l \notin \text{dom}(s)$$

$$\langle s, !l \rangle \hookrightarrow \langle s, s(l) \rangle$$

$$\langle s, l := v \rangle \hookrightarrow \langle s[l \mapsto v], () \rangle$$

Section Section